

Its allow to gracefully handle runtime error
in the python program without crashing program.

only one try block but multiple except block.

try block lets you to test a code for errors.

except block let you to handle error.

finally block lets you execute code
regardless of the result of the try and except.

```
"""
a=5
b=0
#print(a/b)
print("Yo")    # If error occurs IT will not run
"""
```

```
"""
# try
try:
a=int(input())
b=int(input())
print(a/b)
except:
print("error")
"""
```

```
"""
# Zero error
try:
a=int(input())
b=int(input())
print(a/b)
except ZeroDivisionError:
print("error")
"""
```

```
"""
# try -- ZeroDivisionError -- Specified the Zeroerror only
try:
a=int(input())
b=int(input())
print(a/b)
except ZeroDivisionError as z:
print(z)
"""
```

```
"""
# ZeroDivisionError and ValueError -- both are specified
try:
```

```

a=int(input())
b=int(input())
print(a/b)
except ZeroDivisionError as z:
print(f"ZeroDivisionError: {z}")
except ValueError as v:
print(f"ValueError: {v}")

```

'''

#else -- If no error in try , It will run

'''

```

try:
a=int(input())
b=int(input())
print(a/b)
except ZeroDivisionError as z:
print(f"ZeroDivisionError: {z}")
except ValueError as v:
print(f"ValueError: {v}")
else:
print("No Error")
'''

```

'''

finally -- always run

'''

```

try:
a=int(input())
b=int(input())
print(a/b)
except ZeroDivisionError as z:
print(f"ZeroDivisionError: {z}")
except ValueError as v:
print(f"ValueError: {v}")
else:
print("No Error")
finally:
print("See you")
'''

```

'''

interview questions

1 target 9

'''

```

num = [1,2,3,6,7,8]
a=9
l=0
add = 0
r=len(num)-1
while l < r:
add = num[l]+num[r]

```

```

if add == a:
    print([num[l],num[r]])
    l+=1
elif add < a:
    l+=1
else:
    r-=1
'''

#2 op: -- e5d4c3b2a1
'''

a="abbcccddeeeeee"
x={}

```

```

count=0
for i in a:
    if i in x:
        x[i]+=1
    else:
        x[i]=1
y=[]
for i,j in x.items():
    y.append((i,j))
print(y)

```

```

z=sorted(y, key=lambda x:x, reverse=True)
print(z)
print(x)
res = ""
for m,n in z:
    res+=m+str(n)
print(res)

```

```
'''
```

```

#3 2'nd most frequent digit
'''

```

```

a=[5,4,4,6,6,6]
b={}

```

```

for i in a:
    if i in b:
        b[i]+=1
    else:
        b[i]=1
print(b)

```

```

c=[]
for i,j in b.items():
    c.append((i,j))
print(c)

```

```

d=sorted(c, key=lambda x:x[1], reverse=True)
print(d)
print(d[1][0])

```

```
"""
# 4 op == [24,12,8,6]

# all multiple
# and divide by input by increacing 1 index
"""
```

```
x=[1,2,3,4]
m=1
y=[]
for i in x:
    m*=i
for i in x:
    n=m//i
    y.append(n)
print(y)
```

```
"""

# False

##print(0.1+0.2==0.3)
```

```
#
```

```
# Date: 17-03-2026
#5 op -- [3,4,5,1,2] -- change upto input index
"""
```

```
x=[1,2,3,4,5]
y=2
z=x[y:]+x[:y]
print(z)
"""
```

```
# Alternate change upto input index to last
"""
```

```
a=[1,2,3,4,5]
b=int(input())
```

```
for i in range(b):
    x=a.pop(0)
    a.append(x)
print(a)
"""
```

```
# Alternate
```

```
"""

a=[1,2,3,4,5]
b=int(input())
for i in range(len(a)-1):
    if i <= a.index(b+1):
        x=a.pop(0)
        a.append(x)
```

```
print(a)
```

```
'''
```

```
##6 non repeating string to print here -- b
```

```
#
```

```
'''
```

```
x="aabccc"
```

```
def func(a):
```

```
for i in a:
```

```
if a.count(i)==1:
```

```
return i
```

```
print(func(x))
```

```
'''
```

```
# Another Method it s not fully complete
```

```
'''
```

```
a="aabcc"
```

```
for i in a:
```

```
if a.count(i)==1:
```

```
print(i)
```

```
'''
```

```
#7 Missing Sequence Numbers -- op -- [4,6]
```

```
''' '''
```

```
x=[1,2,3,5,7]
```

```
##x=[1,5,9] -- need z=x[i]+1
```

```
y=[]
```

```
z=x[0]
```

```
for i in range(len(x)):
```

```
if x[i] != z:
```

```
y.append(z)
```

```
print(x[i],z,y)
```

```
z=x[i]+1
```

```
''' '''
```

```
###
```

```
#
```

```
Date: 18-03-2026
```

```
#8 longest substring without duplicate values
```

```
'''
```

```
##x="bankaitensazangatsu"
```

```
x="abadbc"
```

```
s=[]
```

```
l=0
```

```
length=0
```

```
for r in range(len(x)):
```

```
while x[r] in s:
```

```
s.remove(x[l])
```

```
l+=1
s.append(x[r])
length=max(length,r-l+1)
print(f"long Sub_str:{length}",s)
print(s)
"""
```

```
## if length>r-l+1:
##     length = length
## else:
##     length = r-l+1
```

```
#9 op -- ["cba","fed"] -- reverse string inside list
"""
```

```
x=["abc","def"]
y=[]
z=""
for i in x:
    for j in i:
        z=j+z
    y.append(z)
    z=""
print(y)
"""
```

```
#10 Duplicate times diferent Values
"""
```

```
a=[1,2,2,1,3,1]
b={}
c=0
for i in a:
    if i not in b:
        b[i]=1
    else:
        b[i]+=1
print(b)
d=list(b.values())
#e=set(d)
e=set(b.values())
```

```
if len(d)== len(e):
    print("True")
else:
    print("False")
"""
```

```
# Same Method but simple for loop
"""
```

```
a=[1,2,2,1,3,1,3]
b={}
c=0
for i in a:
    b[i]=a.count(i)
```

```

print(b)
d=set(b.values())
e=list(b)
if len(d)== len(e):
print("True")
else:
print("False")
'''

```

#11 op – "abcdabef" removal of sequence of repeating strings

```

'''
a="aaabcddabbefa"
b=str(a[0])
for i in range(1,len(a)):
if a[i] != a[i-1]:
b+=a[i]
print(b)
'''

```

Slightly differnece

```

'''
a="aaabcddabbef"
b=""
s=a[0]
for i in range(1,len(a)):
if a[i] != a[i-1]:
s+=a[i]
print(s)
'''

```

#12 op – [[1,1,1],[2,2],[3,3,3],[4,4,4]]

```

'''
x=[1,1,2,4,2,3,1,3,3,4,4]
x.sort()
print(x)
y=[]
z=[x[0]]
for i in range(1,len(x)):

```

```

if x[i] == x[i-1]:
z.append(x[i])
else:
y.append(z)
z=[x[i]]
y.append(z)
print(y)
'''

```

#13 op – [[1,1,1],[4,4],[2,2],[7],[6],[5]]

```

'''

```

```

x=[1,1,1,2,2,4,4,5,6,7,8]
y=[]
z=[x[0]]
for i in range(1,len(x)):

if x[i] == x[i-1]:
z.append(x[i])
else:
y.append(z)
z=[x[i]]
y.append(z)
print(y)

for i in range(len(y)):
for j in range(i+1,len(y)):
if len(y[i])<=len(y[j]) and y[i][0]<=y[j][0]:
y[i],y[j]=y[j],y[i]
print(y)
'''

# Another method
'''

a=sorted(y,key=lambda b:(len(b),b[0]) ,reverse=True)
print(a)
'''

```

#

```

#                                     Date: 19-03-2026
# 14 reverse sort by age
d=[{"name":"urahara","age":23}, {"name":"ichigo","age":24}, {"name":"aizen","age":22}]
'''

```

```

e=sorted(d,key=lambda x:x["age"],reverse=True)
print(e)

```

sort by age

```

f=sorted(d,key=lambda x:x["age"])
print(f)
'''

```

```

##op = True
##a="ababa"
##op = False

```

```

#15 2 palindrome so true
'''

```

```

a="noonmadam"
def palin(a):
for i in range(1,len(a)):
l=a[:i]

```



```

r=a[i:]
if l==l[::-1] and r==r[::-1]:
    return True
return False
print(palin(a))
'''

```

Another Method

```

'''
##a="noonmadam"
a="eveevenoon"      ## --- it is wrong
b=a[0]
c=""
d=""
e=""

```

```

for i in range(1,len(a)):
    if a[i] == a[0]:
        b+=a[i]
        break
    else:
        b+=a[i]
print(b)

```

```

for j in range(len(b),len(a)):
    c+=a[j]
print(c)

```

```

for i in b:
    e=i+e
print(e)

```

```

for j in c:
    d=j+d
print(d)

```

```

if b == e and c == d:
    print("True")
else:
    print("false")
'''

```

#16 op -- [1,2,3,4,5,6]

```

'''
x=[[1,3,5],[2,4,6]]
y=[]
for i in range(len(x[0])):
    y.append(x[0][i])
    y.append(x[1][i])
print(y)

```

```
'''
```

```
#
```

```
#
```

Date: 20-03-2026

#17 op -- 2,12,20,21,22,2 - total how many numbers have 2 , 1 upto 100

```
'''
```

```
a=(input("a: "))
n=int(input("n:"))
b=[]
for i in range(1,n+1):
    if a in str(i):
        print(i)
        #b.append(i)
        #print(f"length : {len(b)} ,\n{b}")
```

```
'''
```

#18 Move zeros to last op -- [1,2,3,4,0,0]

```
'''
```

```
a=[1,0,2,0,3,4]
b=[]
c=[]
for i in a:
    if i != 0:
        b.append(i)
    else:
        c.append(i)
b=b+c
print(b)
```

```
'''
```

Another method

```
'''
```

```
a=[1,0,2,0,3,4]
b=[]
c=a.count(0)
for i in a:
    if i != 0:
        b.append(i)
b.extend([0]*c)
print(b)
```

```
'''
```

#19 Highest value to print -- op 45

```
'''
```

```
a={"a":12,"b":45,"c":23}
b=list(a.values())
m=b[0]
for i in b:
    if i > m:
        m=i
print(m)
```

```
'''
```

```
# Another Method
```

```
'''
```

```
a={"a":12,"b":45,"c":23}
```

```
m=max(a,key=a.get)
```

```
print(m)
```

```
'''
```

```
#20 longest common prefix"
```

```
'''
```

```
a=["flower","flow","flight"]
```

```
p=a[0]
```

```
for i in a[1:]:
```

```
while not i.startswith(p):
```

```
p=p[:-1]
```

```
print(p)
```

```
'''
```

```
b="flow"
```

```
#print(b.startswith("flower")) #-- false
```

```
#21 output valid if all are closed , if not Invalid
```

```
'''
```

```
def is_valid(s):
```

```
mapping = {"(": ")", "{": "}", "[": "]"}
```

```
stack = []
```

```
for char in s:
```

```
if char in mapping:
```

```
top_element = stack.pop() if stack else '#'
```

```
if mapping[char] != top_element:
```

```
return "Invalid"
```

```
else:
```

```
stack.append(char)
```

```
return "Valid" if not stack else "Invalid"
```

```
a = ["(", ")", "[", "]", "{", "}"]
```

```
print(is_valid(a))
```

```
'''
```

```
'''
```

```
def is_valid(s):
```

```
# Map closing brackets to their corresponding opening brackets
```

```
mapping = {"(": ")", "{": "}", "[": "]"}
```

```
stack = []
```

```
for char in s:
```

```
# If it's a closing bracket
```

```
if char in mapping:
```

```
# Pop the top element if stack isn't empty, else use a dummy value
```

```
top_element = stack.pop() if stack else '#'
```

```
# If the popped element doesn't match the required opening bracket
```

```
if mapping[char] != top_element:
```

```
return "Invalid"
```

```
else:
```

```
# It's an opening bracket, push it onto the stack
stack.append(char)

# If the stack is empty, all brackets were matched correctly
return "Valid" if not stack else "Invalid"

# Testing your example
a = ["(", ")", "[", "]", "{", "}"]
# Note: Input usually comes as a string like "()[]{}"
print(is_valid(a))
```

```
'''
```

```
# Another Method failure
'''
```

```
a=["(", ")", "[", "]", "{", "}"]
b=sorted(a)
print(b)
c={}
count=0
for i in b:
    if i in c:
        c[i]+=1
    else:
        c[i]=1
print(c)
d=list(c.values())
if len(d) %2 == 0:
    for i in range(0,len(d),2):
        if d[i] != d[i+1]:
            count+=1
    if count == 0:
        print("Valid")
    else:
        print("Invalid")
    else:
        print("Invalid")

##if "(" in a and ")" not in a :
##    print("False")
##elif "[" in a and "]" not in a:
##    print("False")
##elif "{" in a and "}" not in a:
##    print("False")
##else:
##    print("True")
##else:
##    print("False")
```

```
'''
```

```
#
```

#

Date: 21-03-2026

Saturday -- Holiday -- Ramadhan

#

#

Date: 22-03-2026

Sunday -- Holiday

#

#

Date: 23-03-2026

Mukil Sir -- absent -- Assesment

1. arr=[2,1,8,7,6,3,10,9] -- tar = 10 -- Two and Three sum

"""

a=[2,1,8,7,6,3,10,9]

a.sort()

b=[]

c=[]

target=10

l=0

r=len(a)-1

while l<r:

s=a[l]+a[r]

if s==target:

b.append((a[l],a[r]))

c.append((l,r))

l+=1

elif s>target:

r-=1

else:

l+=1

print(a)

print(b)

print(c)

"""

Three sum

"""

a=[1,4,5,8,7,2]

a.sort()

b=[]

c=[]

t=10

```

l=0
r=len(a)-1
m=1
while l<r:
s=a[l]+a[m]+a[r]
if s==t:
b.append((a[l],a[m],a[r]))
c.append((l,m,r))
l+=1
m+=1
elif s>t:
r-=1
else:
l+=1
m+=1
print(a)
print(b)
print(c)
'''

```

2. second Largest

```

'''
a=[10,20,30,45]
b=a[0]
c=a[1]

for i in a:
if i > b:
b=i
for i in a:
if i > c and i != b:
c=i
print(b," is the Largest Num")
print(c," is the Second Largest Num")
'''

```

3. count char in a string

```

'''
a="Bankai getsuha tenso".lower().replace(" ","")
b={}
for i in a:
b[i] = a.count(i)
print(b)
'''

```

#4. Longest substring

```

'''
a="abacacabd"
b=0
c=0
d=set()
for i in range(len(a)):
while a[i] in d:
d.remove(a[b])

```

```
b+=1
d.add(a[i])
c=max(c,i-b+1)
print(c)
"""
#
```

```
#                               Date: 24-03-2026
# -- file handling --
#
```

```
#                               Date: 25-03-2026
# -- file hndling --
#
```

```
#                               Date: 26-03-2026
# -- half time -- file handling
#
```
